

**Técnicas de Diseño de Algoritmos**  
**Primer Cuatrimestre 2025**  
**2do Recuperatorio**

| Nombre y Apellido | L.U.  | N°Orden |
|-------------------|-------|---------|
| .....             | ..... | .....   |

Turno: ☐ Mañana

☐ Tarde/Noche

Materia: ☐ TDA

☐ AED 3

*Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.*

## 1. Preguntas de opción multiple

*Cada pregunta tiene 4 opciones y una sola opción correcta. Las respuestas correctas (Bien) suman 5 puntos. Las respuestas incorrectas (Mal) restan 2 puntos. Las preguntas no respondidas no se consideran como incorrectas y suman 0 puntos.*

*Marcar en el cuadrado la opción que consideren correcta. Elegir 2 o más opciones se considera una respuesta incorrecta.*

**Pregunta 1** ¿Cuál de las siguientes afirmaciones sobre el árbol generador mínimo T de un grafo G es **falsa**?

- ☐ La arista de menor peso de G, si es única, siempre pertenece a T
- ☐ Preserva las distancias minimax del grafo original
- ☐ Puede calcularse con Prim incluso si G tiene aristas de peso negativas
- ☒ Es único

**Pregunta 2** Dado el dígrafo con 5 nodos y aristas: (1, 2), (1, 3), (2, 4), (3, 4), (4, 5). ¿Cuál de los siguientes es un recorrido de BFS válido?

- ☐ 2 - 1 - 3 - 4 - 5      ☐ 1 - 2 - 4 - 5 - 3      ☒ 1 - 3 - 2 - 4 - 5      ☐ 1 - 2 - 4 - 3 - 5

**Pregunta 3** Dada una implementación del algoritmo de Dijkstra que durante su ejecución elija la versión óptima (es decir, la de menor complejidad) entre las 2 vistas en clase, ¿Cuál es su complejidad computacional para un grafo con N nodos y M aristas?

- ☐  $\Theta(\min\{N^2, M * \sqrt{N}\})$       ☒  $\Theta(\min\{N^2, M * \log(N)\})$       ☐  $\Theta(\min\{N * \log(N), M\})$       ☐  $\Theta(N + M)$

**Pregunta 4** Cuando modelamos un problema con Corte Mínimo, como se vio en clase, ¿Cuál de estas afirmaciones es verdadera?

- ☐ Todas las aristas con pesos finitos van a estar saturadas en la red residual luego de resolver el problema de Corte Mínimo
- ☐ Es posible que una solución valida (al problema original) sea un corte que incluye a S (la fuente) y T (el sumidero) en la misma parte del corte
- ☒ Las aristas con capacidad  $\infty$  no se utilizán en ninguna solución valida (al problema original)
- ☐ El corte está determinado por los nodos alcanzables desde S (la fuente) en el grafo original (es decir, no la red residual)

## 2. Preguntas abiertas de respuesta corta

*Respuesta Bien: +10 puntos. Respuesta Regular: +5 puntos. Respuesta Mal o sin completar: +0 puntos.*

**Pregunta 5** Sean funciones recursivas con las siguientes relaciones de recurrencia en sus complejidades, determinar la complejidad  $\Theta$  de cada una, sin necesidad de demostrar. (0 correctas = Mal, 1 correcta = Regular, 2 correctas = Bien):

- A)  $T(n) = 4 * T(\frac{n}{2}) + n^2 : \Theta(n^2 * \log(n))$  \_\_\_\_\_
- B)  $T(n) = 4 * T(\frac{n}{3}) + n : \Theta(n^{\log_3(4)})$  \_\_\_\_\_

**Pregunta 6** ¿Qué subgrafo produce el algoritmo de Kruskal, para un grafo con N nodos, luego de haber incorporado K aristas a la solución?

Luego de incorporar K aristas a la solución, el algoritmo de Kruskal produce un bosque generador con K aristas (N-K componentes conexas) de peso mínimo.

### 3. Ejercicios a Desarrollar

*Ejercicio Bien: +15 puntos. Ejercicio Regular: +7 puntos. Ejercicio Mal o sin completar: +0 puntos.*

**Pregunta 7** Dado un grafo dirigido con  $N$  nodos y  $M$  aristas ( $M \geq N$ ), ponderado con pesos no negativos, un nodo  $S$  y un número natural  $K$ , diseñar un algoritmo que en  $O(M * K * \log(N))$  determine las longitudes de los  $K$  caminos distintos (no necesariamente simples) más cortos desde  $S$  hasta cada uno de los demás nodos, en orden creciente (de longitud).

Si no hubiera más de  $K$  caminos distintos de  $S$  a un nodo, tiene que producir todas las distancias de caminos distintos desde  $S$  hasta ese nodo.

Para resolver este problema vamos a utilizar una modificación del algoritmo de Dijkstra, en la versión que utiliza una cola de prioridad (heap).

En esta versión:

- Tengo una cola de prioridad de pares (*distancia, nodo*) que guarda posibles formas de llegar a un nodo. En cada paso, devuelve uno de los pares con la mínima distancia
- Al obtener un par (*distancia, nodo*), si el nodo ya fue procesado  $K$  veces, lo descarto y sigo con el siguiente par.
- Si el nodo no ha sido procesado  $K$  veces, la distancia que proceso actualmente es la siguiente forma óptima de llegar a ese nodo, y la agregó a la lista de distancias hacia ese nodo, y registro que fue procesado una vez más.
- Al procesar el nodo  $v$ , para todas las aristas  $(v, u, w)$ , agrego los pares (*distancia + w, u*) a la estructura.
- Inicialmente la estructura solo contiene  $(0, S)$

**Explicación (no era necesaria en el examen):** Cada arista es procesada una vez por cada vez que es procesado el nodo del que sale la arista. Como cada nodo es procesado hasta  $K$  veces, entonces se hacen hasta  $K * M$  procesamientos de aristas. A su vez, al procesar una arista incorporamos un elemento a la estructura, que luego deberemos sacar. Cada uno de estos pasos es  $O(\log(n))$ , por lo cuál la complejidad total es  $O(K * M * \log(n))$

#### Pregunta 8

Una fábrica tiene, hoy,  $E$  empleados,  $M$  máquinas y  $P$  tareas para realizar. La matriz  $EM$  indica si el empleado  $i$  puede utilizar la máquina  $j$  en su celda  $EM[i][j]$ . A su vez, la matriz  $MT$ , en la celda  $MT[j][k]$ , indica si la máquina  $j$  puede utilizarse para resolver la tarea  $k$ .

Para completar una tarea, solo es necesaria una de las máquinas que pueden resolver esa tarea, operada por un trabajador capaz de operar esa máquina, durante un turno laboral.

Cada empleado puede ser asignado a una sola tarea durante su turno y trabaja un solo turno en el día, y cada máquina puede ser utilizada por un operario distinto en cada uno de los 3 turnos que tiene la fábrica.

Los turnos específicos en los que trabajará cada empleado aún no han sido determinados, así que cualquier empleado puede ser asignado a cualquier turno del día, pero solo a uno. Sin embargo, se debe cumplir que no haya más de  $D_i$  empleados trabajando en el  $i$  –ésimo turno. Los empleados a los que no se asigne una tarea no concurrirán a la fábrica en ningún turno.

Se pide modelar esta situación con un grafo dirigido e indicar en él nodos  $S$  y  $T$ , cumpliendo lo siguiente:

- A) El flujo máximo de  $S$  a  $T$  sea igual a la cantidad máxima de tareas que es posible completar en el día.
- B) Cada unidad de flujo se corresponda con la asignación de un empleado y máquina a una tarea, como parte de la solución óptima.
- C) Explicar como recuperar la asignación óptima de empleados a turnos, máquinas y tareas a partir de la solución al problema de flujo.

Sobre este problema hay una consideración importante. Por error de nuestra parte al redactar el enunciado, no es posible hacer un modelo de flujo con los recursos vistos en la materia tal que se cumplan todas las condiciones. Sin embargo, si es perfectamente posible incumpliendo alguna de las siguientes condiciones:

- Que cada máquina no puede usarse más de una vez en un mismo turno
- Que durante el turno  $i$  pueden emplearse hasta  $D_i$  operarios (o lo que es lo mismo, resolverse hasta  $D_i$  tareas)
- Cada tarea puede ser completada una única vez en el día, y no una vez en cada turno.
- Cada operario solo puede trabajar en a lo mucho un turno en el día.

Por lo tanto, si solo incumplen una de estas condiciones, se les dará el ejercicio por Bien, y si incumplen dos de estas condiciones se les dará por Regular (asumiendo que no haya otros errores ni se incumplan otras condiciones del enunciado)

De esta forma, una posible solución correcta, que incumple la primera de las condiciones, sería la siguiente:

Definimos una red de flujo con las siguientes capas:

1. Nodo  $S$  fuente

2. 3 Nodos  $d_l$ . Hay una arista desde  $S$  a cada nodo  $d_l$  ( $0 \leq l < 3$ ) con capacidad  $D_i$ .
3.  $E$  nodos  $e'_i$ . Para cada par  $0 \leq l < 3, 0 \leq i < E$ , hay una arista de capacidad 1 desde  $d_l$  hasta  $e'_i$
4.  $E$  nodos  $e_i$ . Para cada  $0 \leq i < E$  existe una arista de capacidad 1 de  $e'_i$  a  $e_i$
5.  $M$  nodos  $m_j$ . Para cada par  $0 \leq i < E, 0 \leq j < M$ , hay una arista de capacidad 1 desde  $e_i$  hasta  $m_j$  si  $EM[i][j]$  es verdadero.
6.  $M$  nodos  $m'_j$ . Para cada  $0 \leq j < M$ , hay una arista de capacidad 3 desde  $m_j$  hasta  $m'_j$ .
7.  $P$  nodos  $p_k$ . Para cada par  $0 \leq j < M, 0 \leq k < P$ , existe una arista de capacidad 1 desde  $m'_j$  hasta  $p_k$  si  $MT[j][k]$  es verdadero.
8. Nodo  $T$  sumidero. Para cada  $0 \leq k < P$ , hay una arista de capacidad 1 desde  $p_k$  hasta  $T$ .

Notar que esto garantiza que cada máquina se utilice hasta 3 veces, pero no que sean en turnos distintos.

Para recuperar la solución, debemos utilizar cada línea de flujo. Cada línea de flujo pasa por un nodo  $d_l$ , un nodo  $e_i$ , un nodo  $m_j$  y un nodo  $p_k$ . Esto significa que el turno  $l$  el empleado  $i$  utilizará la máquina  $j$  para resolver la tarea  $k$ .

Como hay una relación 1 a 1 entre la asignación de turnos, empleados, máquinas y tareas con las líneas de flujo, maximizar las líneas de flujo maximiza la cantidad de tareas resueltas dadas las restricciones.

### Pregunta 9

Sea el problema:

Dada una lista  $S$  de  $N$  números enteros, determinar la cantidad de inversiones. Es decir, pares  $(i, j)$  ( $0 \leq i < j < N$ ) tales que  $S[i] > S[j]$

Y sea la siguiente solución:

```

1 def Inversiones(S):
2     N = len(S)
3     if N <= 1: return (S, 0)
4     S1, S2 = S[0:(N//2)], S[(N//2):N]
5     (S1, inv_1) = Inversiones(S1) # Llamado recursivo
6     (S2, inv_2) = Inversiones(S2) # Llamado recursivo
7
8     S_ord, cross_inv = Merge(S1, S2)
9     return (S_ord, inv_1 + inv_2 + cross_inv)

```

Donde la función  $Merge(S1, S2)$  recibe 2 vectores ordenados en orden creciente  $S1, S2$  y devuelve en  $\Theta(|S1| + |S2|)$  una tupla donde:

- El primer elemento es  $concat(S1, S2)$  ordenado en orden creciente.
- El segundo elemento es la cantidad de pares  $i, j$  ( $0 \leq i < |S1|, 0 \leq j < |S2|$ ) tales que  $S1[i] < S2[j]$

Demostrar que:

- A)  $Inversiones(S)$  devuelve una tupla  $(S^*, i)$  donde  $S^*$  es el vector  $S$  ordenado e  $i$  la cantidad de inversiones en  $S$ .
- B) La complejidad temporal del algoritmo es  $O(N \log N)$

Cada una de las 2 demostraciones puede estar Bien, Regular o Mal. Para tener el ejercicio Bien ambas deben estar Bien. Para tener el ejercicio Regular se necesita que ambas estén Regular o una Bien. Caso contrario, se tendrá el ejercicio Mal.

a) Realizamos la demostración de forma inductiva:

- Caso base:  $|S| \leq 1$ . En este caso, la función retorna el par  $(S, 0)$ , lo que es correcto porque si  $|S| \leq 1$ ,  $S$  está ordenado, y, por tanto, tiene 0 inversiones.
- Caso inductivo:  $|S| > 1$ . Por tanto, podemos dividir a  $S$  como  $S = S_1 + S_2$  (+ es concatenación de strings). De esta forma, todas las inversiones  $(i, j)$  ( $i < j$ ) cumplen una de las siguientes posibilidades:
  - $i, j < |S_1|$ , por lo cual el par  $(i, j)$  es una inversión dentro de  $S_1$  y está contabilizado en  $inv\_1$  (\*)
  - $i, j \geq |S_1|$ , por lo cual  $(i, j)$  es una inversión en  $S_2$  y se contabiliza en  $inv\_2$ . (\*)
  - $i < |S_1|$  y  $j \geq |S_1|$ , por lo tanto, refieren respectivamente a un elemento de  $S_1$  y un elemento de  $S_2$  tales que  $S_1[i] > S_2[j - |S_1|]$ . Como por hipótesis inductiva, al llamar a  $Merge(S_1, S_2)$  los 2 vectores están ordenados (por ser los resultados de llamar a la función  $Inversiones$ ), entonces esta inversión se contabiliza en  $cross\_inv$

(\*) Por hipótesis inductiva, estos valores son calculados correctamente por los llamados recursivos.

Por lo tanto,  $inv\_1 + inv\_2 + cross\_inv$  son todas las inversiones de  $S$ .

A su vez, como por hipótesis inductiva  $S_1$  y  $S_2$  quedan ordenados,  $S\_ord$  es la concatenación ordenada por la definición de  $Merge$ , y como  $S = S_1 + S_2$ , entonces  $S\_ord$  es el vector  $S$  ordenado.

b) Sea  $n = |S|$ , como:

- $Merge \in \Theta(n)$
- El resto del código también es  $\Theta(n)$
- $S$  se parte en 2 strings de longitud  $\frac{n}{2}$

La complejidad de la función queda  $T(n) = 2 * T(\frac{n}{2}) + \Theta(n)$ .

Por Teorema Maestro,  $T(n) \in \Theta(n * \log(n))$

### Pregunta 10

Sea  $G = (V, E)$  un grafo dirigido, donde cada arista  $e \in E$  tiene una capacidad  $cap(e) > 0$ . Demostrar que el flujo máximo de  $S$  a  $T$  es 0 si y solo si no existe ningún camino dirigido de  $S$  a  $T$  en  $G$ . ( $S, T \in V$ )

Queremos ver que:

1. Flujo máximo = 0 implica que no hay camino de  $S$  a  $T$
2. Que no haya camino de  $S$  a  $T$  implica que el flujo máximo es 0.

Usaremos el teorema de Min Cut - Max Flow, por el cual el valor del máximo flujo es igual al del mínimo corte en el grafo, y puntualmente que el flujo máximo sea 0 es lo mismo a que el corte mínimo sea 0.

1. Que el flujo máximo sea 0 implica que hay un corte con  $S \in A$  y  $T \in B$  tal que no hay ninguna arista desde un nodo en  $A$  a un nodo en  $B$ , porque si hubiera una el corte tendría un costo mayor a 0. Por tanto, no hay ningún camino de ningún nodo de  $A$  a ningún nodo de  $B$ , y particularmente no hay ningún camino de  $S$  a  $T$ .

2. Consideremos el conjunto  $S'$  de nodos alcanzables desde  $S$ . Como no hay camino de  $S$  a  $T$ ,  $T \notin S'$ . A su vez, por transitividad, ninguno de estos nodos puede alcanzar ningún nodo que esté en  $S'$ , y particularmente no existe ninguna arista desde un nodo  $u \in S'$  a un nodo  $v \in (V - S')$ . Por tanto, existe un corte de costo 0 dado por  $A = S'$  y  $B = (V - S')$ , con  $S \in A$  y  $T \in B$ .